

# OpenAI Codex 最新主线与当前定制版差距分析报告

jqwang / Codex

2026 年 3 月 15 日

## 摘要

本文基于 [OpenAI Codex 官方仓库](#) 当前最新主线，以及本地当前定制版 `main` 分支的实际代码状态，对两者的功能差距、架构漂移和合并风险进行一次决策导向的梳理。

结论不是“官方更先进，所以直接升级即可”，而是：**两边已经分叉成两条产品线**。官方在 `app-server v2`、插件/应用生态、审批治理、实时链路、工具协议标准化等通用平台能力上明显领先；当前定制版则在 `macOS` 本机能力、定制安全放行、工作流型子代理、多模型路由、`Entire` 集成和定制 `TUI` 体验上更强。未来若要缩小差距，应按功能族逐步迁移，而不应直接把整条分支硬合并。

## 目录

|                                                                 |          |
|-----------------------------------------------------------------|----------|
| <b>1 执行摘要</b>                                                   | <b>3</b> |
| 1.1 一句话结论                                                       | 3        |
| 1.2 核心判断                                                        | 3        |
| <b>2 比较基线与方法</b>                                                | <b>3</b> |
| 2.1 比较对象                                                        | 3        |
| 2.2 为什么本报告以 <code>main</code> 为主，不以 <code>release tag</code> 为主 | 3        |
| 2.3 量化基线                                                        | 3        |
| 2.4 本报告的方法                                                      | 4        |
| <b>3 官方主线相比当前定制版的主要新增能力</b>                                     | <b>4</b> |
| 3.1 <code>App Server v2</code> 的文件系统 <code>RPC</code> 更完整       | 4        |
| 3.2 <code>Plugin / App</code> 生态继续扩展                            | 4        |
| 3.3 动态工具可见性继续标准化                                                | 4        |
| 3.4 <code>TUI</code> 与 <code>app-server</code> 的一体化继续增强         | 4        |
| 3.5 <code>Smart Approvals / GuardianSubagent</code> 审批流         | 5        |
| 3.6 <code>Realtime v2 / Transcription / Handoff</code> 继续推进     | 5        |
| 3.7 <code>Provider</code> 配置治理更规范                               | 5        |
| 3.8 多代理协议正在收敛为更稳定的上游契约                                          | 5        |
| <b>4 当前定制版相比官方主线的独有优势</b>                                       | <b>5</b> |
| 4.1 <code>macOS Endpoint Security</code> + 人工批准式安全放行            | 5        |
| 4.2 <code>/freeze</code> 时间冻结 / 快照调试工作流                         | 5        |

|                                              |   |
|----------------------------------------------|---|
| 目录                                           | 2 |
| 4.3 子代理隔离 worktree 的 lease / resume 机制       | 6 |
| 4.4 model_sub / utility_model 多模型路由          | 6 |
| 4.5 Entire 集成、摘要生成与 Context Packet           | 6 |
| 4.6 MemoryLink 贯通到 app-server v2 / hooks     | 6 |
| 4.7 TUI Git Graph / Session Bar / Ralph Loop | 6 |
| 5 差距判断：不是简单的新旧关系，而是能力重心不同                    | 7 |
| 5.1 官方主线更强的方向                                | 7 |
| 5.2 当前定制版更强的方向                               | 7 |
| 5.3 真正的风险点                                   | 7 |
| 6 合并风险与冲突热点                                  | 7 |
| 6.1 高风险冲突区域                                  | 7 |
| 6.2 为什么不建议直接整分支合并                            | 8 |
| 7 建议的迁移优先级                                   | 8 |
| 7.1 优先吸收的官方能力                                | 8 |
| 7.2 应优先保留的当前定制能力                             | 8 |
| 7.3 建议的工程路线                                  | 8 |
| 8 最终结论                                       | 8 |

## 1 执行摘要

### 1.1 一句话结论

当前定制版并不是“单纯落后几个版本”，而是与官方最新主线发生了大规模双向分叉。

### 1.2 核心判断

- 从主线演进看：官方已经明显向“标准化平台 + app-server 协议 + 审批治理 + 插件/应用生态”继续推进。
- 从差异结构看：当前定制版保留了多项官方没有的高价值特性，尤其是面向 macOS、本地安全控制、工作流型协作和多模型运营侧能力。
- 从集成成本看：直接整分支合并的风险很高，冲突会集中在四大区域：core、tui、app-server、app-server-protocol。
- 从策略上看：更合理的路径是“以最新上游为底座，按功能族回迁定制能力”，而不是继续在当前大分叉分支上滚动同步。

## 2 比较基线与方法

### 2.1 比较对象

- 官方仓库：<https://github.com/openai/codex>
- 本地定制版：当前工作区 main
- 官方最新主线基线：FETCH\_HEAD，对应上游 main 最新提交

### 2.2 为什么本报告以 main 为主，不以 release tag 为主

- 截至 2026 年 3 月 15 日，GitHub Releases 页面顶部显示最新预发布版本。
- 该版本号为 rust-v0.115.0-alpha.24。
- 但本次要分析的是“官方现在已经走到哪里”，因此以官方 main 主线更能反映真实功能差距。
- 官方当前主线最新提交为 49edf311，提交时间为 2026-03-14 22:24:13 -0700。

### 2.3 量化基线

| 对象       | 标识       | 说明                                                                   |
|----------|----------|----------------------------------------------------------------------|
| 官方主线最新提交 | 49edf311 | 2026-03-14 22:24:13 -0700，提交标题为 [apps] Add tool call meta. (#14647)  |
| 当前定制版主线  | 112cd1b4 | 2026-03-15 18:26:42 +0800，提交标题为 Merge branch 'feature/upstream-sync' |
| 共同祖先     | 79d6f80e | 2026-02-26 14:11:10 +0000，说明两边自 2 月底之后已发生大规模双向演进                     |

- 自共同祖先以来，官方主线独有提交约 496 个。
- 自共同祖先以来，当前定制版独有提交约 175 个。

- 直接内容差异约为：**1587 个文件变更**，**+215149 / -129182 行**。
- 这些数字只能说明漂移规模，**不能直接等价于功能差距**，因为当前定制版已经手工吸收、改写或重实现过部分上游能力。

## 2.4 本报告的方法

1. 先用 Git 基线确认真正的最新上游主线；
2. 再按功能域比较，而不是只看 commit 数；
3. 对“官方已有、当前定制版尚未吸收”的能力做重点梳理；
4. 对“当前定制版独有、官方无直接替代”的能力单独保留；
5. 最后给出迁移优先级，而不是给出误导性的“建议直接同步”。

# 3 官方主线相比当前定制版的主要新增能力

## 3.1 App Server v2 的文件系统 RPC 更完整

官方主线已经补齐 app-server v2 的文件系统操作面，包括 `fs/readFile`、`fs/writeFile`、`fs/createDirectory`、`fs/readDirectory`、`fs/copy`、`fs/remove` 等。这意味着官方正在把 app-server 明确推进为一个更标准的“远程编排接口层”，而不是仅服务于本地 TUI。

当前定制版里缺少对应的本地实现文件，例如本地并不存在 `codex-rs/app-server/src/fs_api.rs`。这类差距不是小修小补能补平的，而是接口面和产品定位已经发生了变化。

## 3.2 Plugin / App 生态继续扩展

官方主线已经有更完整的 `plugin/read` 能力和对应 schema，说明插件/应用发现、元数据读取、能力暴露这条链路正在继续扩充。官方最新主线里可以直接看到 `plugin/read` 出现在 app-server protocol 中，而当前定制版尚未接上这条协议面。

如果未来要做更完整的 `skills / plugin / app` 混合生态，官方方向显然走得更远。

## 3.3 动态工具可见性继续标准化

官方已经将动态工具“是否暴露给上下文”的能力做进协议，字段名是 `exposeToContext`。这类能力的价值在于：工具不再只是“能不能调用”，而是可以更精细地控制“模型当前能不能感知到这个工具”。

当前定制版没有这套协议字段和行为链路，因此在工具编排的可治理性上略显粗糙。

## 3.4 TUI 与 app-server 的一体化继续增强

官方主线中已经存在 `codex-rs/app-server/src/in_process.rs`，表明它进一步向“在 TUI / app / runtime 里内嵌 app-server”推进。配合 richer initialize platform fields，这说明上游在为多前端、多宿主、多进程形态做一致化。

当前定制版没有这条集成线，因此如果目标是和官方 app 生态对接，后续会越来越吃亏。

### 3.5 Smart Approvals / GuardianSubagent 审批流

这是本次最值得关注的新能力之一。官方主线已经将审批流从“纯人工弹框确认”推进到“由专门审查子代理做风险判断再决定是否放行”。

其核心逻辑不是放宽权限，而是把 `on-request` 审批交给一个只读、不可再次提权的 `guardian` 子代理进行风险评估。它会：

- 重建与该审批有关的上下文；
- 审查精确的计划动作 JSON；
- 输出 `low / medium / high` 风险和 0-100 风险分；
- 在风险分低于阈值时自动批准，否则拒绝或继续要求人工确认。

这套设计的产品意义很强：它不是提权，而是减少无意义的人类确认次数，同时把风险门槛做成可审计的结构化过程。

### 3.6 Realtime v2 / Transcription / Handoff 继续推进

官方主线在 Realtime v2 parser、transcription 和 handoff 相关链路上仍在快速推进。当前定制版并未对齐这一轮更新，因此如果未来目标包括更标准的实时语音、实时会话编排或语音接管能力，当前定制版会明显落后。

### 3.7 Provider 配置治理更规范

官方主线对 provider / config 的治理比当前定制版更标准化，例如更明确的 `openai_base_url` 配置边界、内建 provider 不应被随意覆盖等。当前定制版在 provider 路线上更偏“运营型定制”，灵活但不够统一。

这意味着：当前定制版更适合内部定制与多通道运营，但官方更适合做长期可维护的平台能力。

### 3.8 多代理协议正在收敛为更稳定的上游契约

官方主线在 multi-agent prompt、handoff、事件元数据、角色约束上持续迭代。当前定制版并不是没有多代理，而是有一套自己的扩展语义，因此这里不是“有无”的差距，而是**契约已经分叉**。

从长期维护看，越晚对齐，后续 app-server v2、TUI、多前端和自动化工具链的兼容成本越高。

## 4 当前定制版相比官方主线的独有优势

### 4.1 macOS Endpoint Security + 人工批准式安全放行

当前定制版最有代表性的差异化能力之一，是围绕 macOS Endpoint Security 的本机安全拦截与放行工作流。其意义不在于“多一个工具”，而在于把系统级安全事件与代理执行流程联动起来。

这类能力属于强平台绑定、本机控制链路型能力，官方主线目前没有等价替代。

### 4.2 /freeze 时间冻结 / 快照调试工作流

当前定制版增加了 `/freeze` 调试路径，可以将问题现场冻结成可分析快照，用于回放和排障。这类能力更接近“复杂代理工作流的法医式调试工具”，官方主线没有直接同类物。

### 4.3 子代理隔离 worktree 的 lease / resume 机制

当前定制版对子代理和线程切分的处理更偏工程执行流：不仅能 spawn 子代理，还可以把它们绑定到独立 worktree，并对 worktree 与 thread 的关系做 lease / resume 持久化。

这对复杂多分支、多任务并行开发非常有价值，而官方主线当前更偏通用协作协议，不强调这类本地 Git 工作流深度绑定。

### 4.4 model\_sub / utility\_model 多模型路由

当前定制版有明显强化过的多模型调度能力，包括：

- model\_sub / utility\_model;
- provider account pool;
- auth key fallback;
- 多轮 failover;
- 对 Antigravity、Gemini、Grok、Gemma 等 provider / model 的扩展支持。

这使当前定制版在“多模型运营和资源池调度”上比官方更灵活，但这也是它与上游代码结构冲突最重的部分之一。

### 4.5 Entire 集成、摘要生成与 Context Packet

当前定制版将 Entire 相关工作流、AI 摘要生成、上下文封包等能力做成了一套较完整的增强链路。这种能力更偏内部工作流优化，强调的是信息汇总、上下文压缩和后续协作效率。

官方主线目前没有等价的 Entire 集成，因此这里属于当前定制版的真实差异化资产。

### 4.6 MemoryLink 贯通到 app-server v2 / hooks

当前定制版已经把 MemoryLink 传播到 app-server v2、hooks 和事件层。需要注意的是，这里不能简单说“官方没有 memory，定制版有”。更准确的说法是：**两边都在做 memory，但 wire contract 和集成路径不一样。**

因此后续若要吸收上游 app-server v2 新能力，memory 相关结构大概率需要重新对齐。

### 4.7 TUI Git Graph / Session Bar / Ralph Loop

当前定制版的 TUI 有明显更强的“工作台化”倾向，例如：

- Git Graph;
- Session Bar;
- Ralph Loop;
- 与历史会话、子代理、仓库状态关联更紧的交互方式。

官方主线在通用交互和平台收敛上更强，但当前定制版在“重度工程使用场景”下的体验更有特色。

## 5 差距判断：不是简单的新旧关系，而是能力重心不同

### 5.1 官方主线更强的方向

- 更标准的 app-server v2 协议面；
- 更完整的 plugin / app 生态基础；
- 更细的动态工具治理；
- 更成熟的审批治理链路；
- 更持续演进的 realtime / transcription 能力；
- 更适合多前端、多宿主、多客户端的长期平台化。

### 5.2 当前定制版更强的方向

- 更强的 macOS 本机安全与执行控制；
- 更强的工程执行工作流；
- 更灵活的多模型与账号池调度；
- 更深的内部工具与上下文增强；
- 更偏“高强度开发者工作台”的 TUI 使用体验。

### 5.3 真正的风险点

真正的风险不在于“官方有没有比我们多一些 commit”，而在于：

- 上游平台能力仍在快速演进；
- 当前定制版在若干核心区域深度改过；
- 这导致后续每一次同步都更像一次重新集成，而不是普通 rebase。

## 6 合并风险与冲突热点

### 6.1 高风险冲突区域

| 区域                               | 风险说明                                                       |
|----------------------------------|------------------------------------------------------------|
| codex-rs/core/src                | 同时承载模型调度、审批、memory、子代理、sandbox、工具路由等核心行为，是冲突密度最高的区域。       |
| codex-rs/tui/src                 | 当前定制版对 TUI 的产品化增强较多，而官方主线也在持续重构交互与 app-server 对接，容易出现行为回归。 |
| codex-rs/app-server/src          | 官方在这里明显前进更多，尤其是文件系统 RPC、plugin/app 链路、进程内启动方式。             |
| codex-rs/app-server-protocol/src | wire contract 漂移非常明显，任何粗暴合并都可能造成 schema、TS 生成物和客户端行为不一致。   |
| schema 生成物                       | 即使业务逻辑合并成功，schema 生成物也容易大面积漂移，放大 review 难度。                |

## 6.2 为什么不建议直接整分支合并

- 历史上已经发生过多轮大规模上游同步，旧冲突解决逻辑会继续污染新的同步；
- 当前定制版并不只是局部 patch，而是形成了自己的产品逻辑；
- 直接整分支 merge 会把“有价值的差异化能力”和“过时的兼容层”混在一起带回主线，难以治理。

# 7 建议的迁移优先级

## 7.1 优先吸收的官方能力

1. app-server v2 文件系统 RPC；
2. plugin/read 与更完整的 plugin / app 元数据读取；
3. exposeToContext 这类动态工具可见性治理；
4. Smart Approvals / GuardianSubagent 审批流；
5. provider 配置治理和更清晰的 config 边界。

## 7.2 应优先保留的当前定制能力

1. macOS Endpoint Security + 安全放行；
2. /freeze 快照调试；
3. 子代理独立 worktree 与 lease / resume；
4. model\_sub / utility\_model / account pool / failover；
5. Entire 集成、上下文封包、定制摘要链路；
6. Git Graph / Session Bar / Ralph Loop。

## 7.3 建议的工程路线

1. 以官方最新 main 为新底座建立干净分支；
2. 按功能族逐块回迁当前定制能力，而不是整分支回放；
3. 先处理协议和 app-server，再处理多模型和 memory，再处理 sandbox / TUI 深度定制；
4. 每一族能力都单独验证，不把“看起来能编译”误当作完成。

# 8 最终结论

当前定制版和官方最新主线之间，已经不是“一边先进一边落后”的线性关系，而是“官方平台化继续推进，而当前定制版沿着本地工程 workflow 深度定制”的双轨结构。

如果目标是：

- 对齐官方生态，就要优先吸收 app-server v2、plugin/app、审批治理、动态工具治理；
- 保住当前产品特色，就必须把 macOS 安全、工作流型子代理、多模型路由、Entire 集成和定制 TUI 作为明确资产保留；
- 控制长期维护成本，就不应再把未来同步当成一次次大 merge，而应改为“上游底座 + 定制能力分层回迁”的路线。



换言之, 差距确实已经很大, 但并不是单方向的落后; 真正需要的是一份 keep / adopt / drop 迁移矩阵, 而不是一次粗暴升级。

## 附录: 代表性参考链接

- 官方仓库: <https://github.com/openai/codex>
- 官方最新主线提交: <https://github.com/openai/codex/commit/49edf311ac3ae84659b0ec5eacd5e471c881eee8>
- 共同祖先提交: <https://github.com/openai/codex/commit/79d6f80e41806d61b8a5ce7dbaff4bb1d6e38a91>
- Smart Approvals: <https://github.com/openai/codex/commit/bc24017d64829d0b97b8bc6ed529a389e1e8bc1b>
- app-server FS API: <https://github.com/openai/codex/commit/f8f82bfc2b558229cc4f7ef6245c474ee8b389c7>
- 动态工具可见性: <https://github.com/openai/codex/commit/70eddad6b075f26f0f93c66f7ec9a4e49cdadc93>
- TUI 内嵌 app-server: <https://github.com/openai/codex/commit/9dba7337f21dbc720bd5af70c1628d7c3217f47b>
- Provider 配置治理: <https://github.com/openai/codex/commit/4b9d5c8c1bdb6d9cfd43570e0b8e88c88b54d823>
- 多代理契约收敛: <https://github.com/openai/codex/commit/36dfb844277e79793766f96305c9633f90bc043e>
- Realtime v2 parser: <https://github.com/openai/codex/commit/3e8f47169e523d2004fe4491bd2e29e78f9c6720>